

Constrained Edge Pruning and Trust-Aware GNN Inference for Task Allocation in Dynamic Rover Networks: a QUBO Formulation with a Solver-Level Audit

Anonymous Authors
quantum-gnn project

September 14, 2026

Abstract

Mobile rover swarms communicate over wireless links whose quality changes constantly, so the graph describing the swarm is dynamic. We study a pipeline in which a constrained quadratic unconstrained binary optimization (QUBO) model prunes the dynamic graph each cycle and a trust-aware message-passing GNN assigns tasks on the pruned graph, with trust updated online from observed outcomes. Rather than only proposing the formulation, we audit it. First, we verify the QUBO encoding against an independent re-derivation and exhaustive enumeration. Second, we show that penalty-encoded constrained QUBOs are difficult for annealing-style samplers: local bit-flip simulated annealing, a two-bit-move variant, and the **dwave-neal** sampler reach the exact optimum in only 52 to 60% of verifiable instances, while a feasibility-projected annealer that moves over valid edge sets solves 100% of them, and 95% against exact MILP optima on larger instances (mean true optimality gap below 10^{-3}). Third, we show the uncomfortable counterpoint: the same problem solved directly as a mixed-integer program by CBC is exact and roughly 100 times faster than our annealer, so on classical hardware the QUBO’s role is as a hardware-target encoding, not as the recommended solver. End-to-end, trust-aware assignment improves mission success over random assignment by 13.8 percentage points and over a trust-only baseline by 8.1 points (paired bootstrap 95% CI [+4.2, +11.7], permutation $p = 0.016$), with mission success now causally dependent on the pruned topology. We report negative results with equal prominence: the QUBO pruner does not beat a threshold pruner on raw success at ten-rover scale, message passing adds a statistically insignificant margin over a no-graph readout, and a cloud-trained readout does not improve over fixed weights.

1 Introduction

Teams of small rovers coordinate over low-power radios (in our case ESP-NOW between ESP32-class nodes), which makes the communication graph highly dynamic: links appear, degrade, and vanish as rovers move, and nodes vary in reliability as batteries drain and sensors fault. Allocating tasks over such a graph raises two problems. First, running graph neural network (GNN) inference on the full, messy graph wastes fog-tier compute and lets degraded edges distort decisions. Second, naive pruning cannot respect global constraints such as degree limits, processing budgets, and the requirement that mission-critical nodes stay connected.

This paper studies a pipeline that prunes the dynamic graph with a constrained QUBO model and then performs trust-aware GNN inference on the surviving subgraph. Our intent from the start was to make the system quantum-ready: the QUBO form is what quantum and quantum-inspired annealers consume. What the work actually demonstrates, however, is broader and more useful: a careful account of when such a formulation is solved well, when it is not, and what each component of the pipeline contributes. Concretely, we contribute:

- A constrained QUBO formulation for dynamic graph sparsification with utility, stability, degree, budget, and connectivity terms, connectivity enforced exactly by a single-commodity flow encoding inside the QUBO (Section 4), with the implementation validated against an independent re-derivation and exhaustive enumeration.
- A solver-level audit of that encoding (Section 5): bit-level annealers (including `dwave-neal`) solve only about half of exactly verifiable instances due to a penalty-barrier effect we quantify, while a feasibility-projected annealer solves all of them, and an exact MILP baseline (CBC) solves the problem directly, faster than any annealer.
- A trust-aware assignment layer with an online EWMA trust model, aging rehabilitation, and explicit ablations isolating what the GNN contributes over trust-only and no-graph baselines, with paired statistics (Section 6).
- An end-to-end evaluation in which mission success depends causally on the pruned topology, including pruning baselines, latency, staleness under cloud-amortized solving, fault recovery, sensitivity analysis, and negative results (Section 7).

2 Related Work

Graph neural networks were popularized by Kipf and Welling [1] and extended to inductive settings by Hamilton et al. [2]; we use a lightweight trust-weighted message-passing variant suited to fog devices. QUBO and Ising formulations are surveyed by Glover et al. [3] and Lucas [4]; simulated annealing dates to Kirkpatrick et al. [5] and is available in production form through D-Wave’s Ocean SDK [6]. Our exact classical baseline uses the CBC branch-and-cut solver from COIN-OR [7], and our MILP encoding exploits the integrality of single-commodity flow relaxations, a standard result in network optimization [8]. Multi-robot task allocation is surveyed by Korsah et al. [9]; trust models for multi-agent systems are surveyed in [13]. When trust estimation is isolated from the graph, the assignment problem reduces to a bandit problem [10]; we include the corresponding trust-only baseline explicitly, since it is the strongest simple competitor. Graph sparsification has a large classical literature [11]; our setting differs in requiring online, constraint-carrying decisions on a dynamic graph. The mist/fog/cloud split follows Bonomi et al. [12]. To our knowledge, the combination of constrained QUBO pruning with online trust-aware GNN inference for rover task allocation, and in particular the solver-level comparison of penalty-encoded QUBO, feasibility-projected annealing, and direct MILP for this problem, has not been reported before.

3 System Model

The deployment follows a three-tier architecture (Fig. 1). Rovers (mist tier, ESP32-class) sense their state and exchange telemetry over ESP-NOW. A gateway forwards telemetry to a Raspberry Pi (fog tier), which maintains the dynamic graph, prunes it, and runs assignment. A laptop or server (cloud tier) retrains the readout and can amortize expensive solves.

At cycle t the swarm is an undirected graph $G_t = (V, E_t)$. Each edge $e = (i, j)$ carries normalized features $(T_e, L_e, B_e, P_e, A_e, C_e, c_e) \in [0, 1]^6 \times \mathbb{N}$: trust $T_e = \min(T_i^{t-1}, T_j^{t-1})$ from the previous cycle’s trust model, link quality L_e , battery $B_e = \min(B_i, B_j)$, proximity P_e , freshness $A_e = \exp(-\Delta t_e / \tau)$ from the last successful observation of the link, task criticality C_e from the tasks running at the endpoints, and a processing cost that models transmission and queueing time, $c_e = \max(1, \lfloor 10 \bar{c}_e \rfloor)$ with $\bar{c}_e = 0.1 + 0.5(1 - P_e) + 0.15(\ell_i + \ell_j)$. Node features are trust T_i , battery B_i , load ℓ_i , and capability q_i .

Mission success is causal through the pruned graph. An assigned rover *executes* a task successfully with probability equal to its hidden reliability (gated by battery); execution outcomes drive

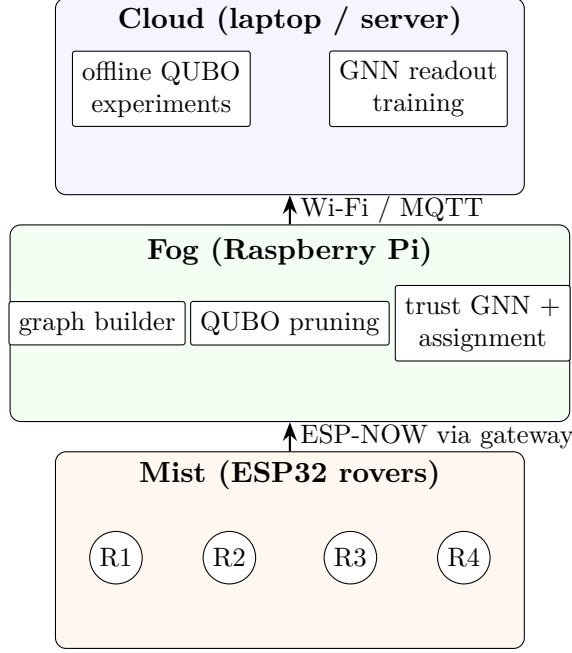


Figure 1: Three-tier deployment. Telemetry flows up; task assignments flow down. Expensive solving and training are cloud or fog functions depending on the cycle budget.

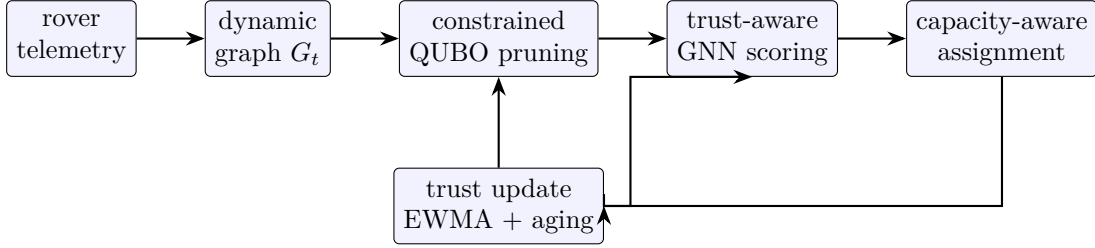


Figure 2: One pipeline cycle. Trust from the previous cycle feeds both the QUBO edge features and the GNN node features, closing the loop without same-cycle circularity.

trust updates. The task result must then be *delivered* to the root rover over the kept subgraph, succeeding with probability equal to the weakest link quality on the node’s widest path to the root. Mission success is the conjunction. This matters: in an earlier version of our simulator, success depended only on node reliability, and the entire pruning layer was causally disconnected from the metric it was supposed to improve. We consider fixing that coupling a prerequisite for any end-to-end claim in this paper.

Each cycle executes the pipeline of Fig. 2: build the graph, prune edges subject to constraints, score nodes, assign tasks greedily under per-node capacity and resource gates, observe outcomes, and update trust. A fixed mission-critical node set (chosen once per scenario, not redefined per cycle) anchors the connectivity constraint.

4 Constrained QUBO Formulation

Let $O = E_t \setminus H$ be the optional edges and H the hard-critical edges fixed to 1. Let R be the required node set with root r , $m = |R| - 1$, degree caps D_i , and global cost budget K . With binary decision variables x_e for optional edges, binary-encoded integer slacks s_i, s_C , and integer flow variables f_{uv} , the energy is

$$\begin{aligned}
H(z) = & \sum_{e \in O} (-U_e + \lambda) x_e + \eta \sum_{e \in O} (x_e - x_e^{t-1})^2 \\
& + \rho_d \sum_{i \in V} \left(\sum_{e \in \delta_O(i)} x_e + s_i - (D_i - \deg_H(i)) \right)^2 \\
& + \rho_C \left(\sum_{e \in O} c_e x_e + s_C - (K - c_H) \right)^2 \\
& + \rho_F \sum_{v \in V} (\text{in}_v - \text{out}_v - b_v)^2 + \rho_{FC} \sum_{e \in E} (f_{uv} + f_{vu} + q_e - m x_e)^2, \tag{1}
\end{aligned}$$

where U_e is the weighted edge utility with weights summing to one, λ is a sparsity penalty, η a topology-stability penalty, $b_r = -m$, $b_v = 1$ for $v \in R \setminus \{r\}$ and 0 otherwise, and $x_e := 1$ for $e \in H$. The flow terms enforce exactly that the required set remains connected in the selected subgraph.

The penalties satisfy $\rho_d = \rho_C = 100$ and $\rho_F = \rho_{FC} = 500$ against a soft objective of magnitude at most about 1.2 per edge. Each unit of constraint violation costs at least 100 but unlocks at most one edge worth at most 1.2, and flow violations are quadratically penalized against a linear gain, so no violation can be profitable at any problem size. We confirmed this exhaustively: over 38 randomly generated instances, the exact bit-level optimum of Eq. (1) never contains a violated constraint.

We want to be precise about the strength of our verification claim, because “proven correct” is easy to overstate. The encoding is derived from the specification above; the implementation is then validated empirically against an *independent* re-derivation of Eq. (1): on 40 instances with 8 random states each, the matrix energy $z^\top Q z + \text{const}$ agrees with the independent evaluation to within 2.3×10^{-10} . This is strong empirical validation of the implementation, not a formal proof, and we report it as such.

5 Solver Analysis: The Penalty-Barrier Problem

A QUBO like Eq. (1) is meant to be sampled by an annealer. We found that this fails in a specific, quantifiable way, and we are careful to scope the claim to what we measured. From any feasible state, every single-bit flip either breaks a constraint (penalty at least 100) or changes the soft objective by at most about 1.2. At temperatures useful for the soft landscape, the acceptance probability of a constraint-breaking move is effectively zero. We therefore expected single-bit simulated annealing to be unable to move between feasible regions, and it cannot: random restarts returned infeasible states in 19 of 25 test instances. The natural objection is that this is an artifact of the move set, so we also tested a two-bit-move variant and a production sampler (**dwave-neal**, 100 reads). Both improve matters only marginally (Fig. 3). Our claim is therefore narrow: *on this penalty-encoded landscape, local QUBO samplers of the annealing family perform poorly*, not that all QUBO methods must fail.

Our solver, *projected edge annealing*, moves in the space of feasible edge subsets instead of the space of bit strings. A move toggles one optional edge or swaps two edges, and the candidate is projected back to feasibility by an exact repair procedure that recomputes slacks and a spanning-tree flow; unprojectable moves are rejected. The objective is still exactly $H(z)$, restricted to the feasible manifold where the penalty terms vanish; the QUBO remains the specification, and the Q matrix is still exported for hardware backends. A deterministic polish alternates single-toggle and pairwise-swap passes to a local fixpoint, and random restarts are seeded with degree-cap-aware random spanning trees, because greedy edge-by-edge growth can never cross the infeasible prefix states that connectivity constraints create. Each solve uses about 2.4×10^4 energy evaluations and about 2.1 s on our benchmark instances.

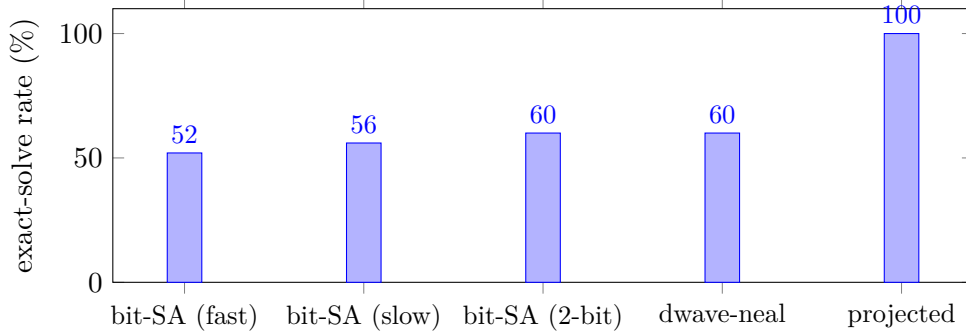


Figure 3: Exact-solve rate on 25 instances with exhaustively enumerated ground truth. The 2-bit variant and `dwave-neal` sample the same Q matrix with richer effort and still miss two in five instances.

5.1 The strong classical baseline: direct MILP

The fairest question to ask of a QUBO pipeline is why not solve the original constrained problem directly. The soft objective of Eq. (1) is linear in x , the stability term is linear for binary variables, and the single-commodity flow relaxation is integral, so the problem is a mixed-integer linear program. We solve it exactly with CBC [7]. On 20 instances with up to 40 optional edges, the projected annealer reaches the MILP optimum in 95% of cases (mean normalized gap 7×10^{-5} , worst 1.4×10^{-3}), using the normalized gap $(H_{\text{SA}} - H^*) / \max(1, |H^*|)$. On 25 small instances it remains 100% exact against exhaustive enumeration, and on 20 larger instances (up to 817 binary variables) its gap against the best of four annealer seeds is 6.9% on average.

The counterpoint is just as important: CBC solves these instances in about 15 ms, against about 2.1 s for the annealer. On classical hardware, direct MILP is both exact and roughly 100 times faster. What remains for the QUBO encoding is precisely its original purpose: it is the input format of quantum and quantum-inspired annealing hardware, and the projected annealer is the strong classical stand-in until such hardware is evaluated. We state this plainly rather than overselling the formulation.

6 Trust-Aware GNN Inference

On the pruned graph, each node is scored for each task by a two-layer trust-weighted message-passing network. With $h_i^{(0)} = [T_i, B_i, 1 - \ell_i, q_i]$,

$$m_i = \frac{\sum_j a_{ij} h_j}{\sum_j a_{ij}}, \quad h'_i = \text{ReLU}(W_s h_i + W_n m_i + b), \quad s_i = \sigma(w^\top h_i^{(2)} + \beta),$$

where $a_{ij} = T_{ij} L_{ij}$ weights messages by link trust and quality. The self-weight (0.8) dominates the neighborhood weight (0.2); nodes below per-task trust, battery, or compute thresholds are gated to zero, with a deadlock fallback that relaxes the trust gate when it would leave a task unassigned. Scores are modulated by $(0.5 + 0.5 p_i)$, where p_i is the weakest-link quality on node i 's path to the root, so the assigner prefers strong delivery paths without letting the root absorb every task.

Trust itself is dynamic. After each cycle, an assigned node's trust follows an EWMA, $T_i \leftarrow \alpha T_i + (1 - \alpha) \text{obs}_i$, while unobserved nodes age toward the prior, $T_i \leftarrow T_i + \delta(0.5 - T_i)$. Aging rehabilitates a rover after a single unlucky failure; without it we observed permanent blacklisting, and without per-task load feedback we observed overload lockout. Hidden reliability in the simulator drifts over time and is spatially correlated (nearby rovers share conditions), so the trust model must track a moving, structured target rather than estimate a stationary Bernoulli mean. The readout (w, β) can be trained offline by logistic regression on collected outcomes; we collect training data with a high exploration rate so that a random-policy, selection-bias-free subset exists (Section 7.4).

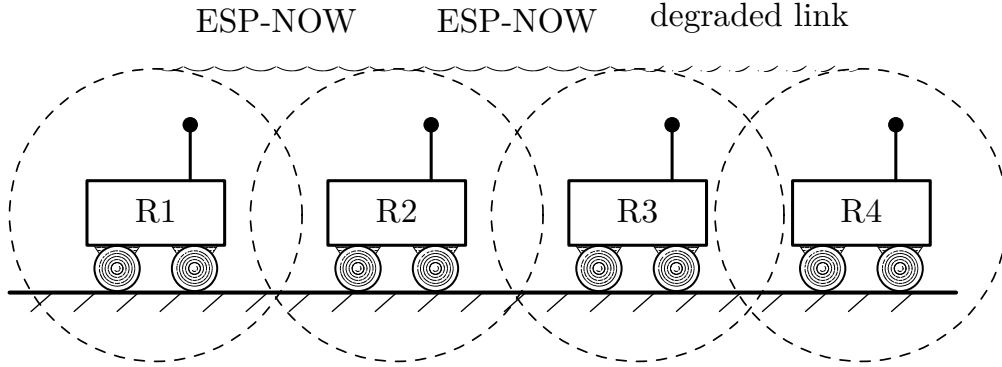


Figure 4: Physical testbed sketch (fiziko rendering when available, TikZ fallback otherwise). Four rovers on a flat surface with per-link quality arcs; the degraded link is a pruning candidate.

Table 1: Pruner comparison, 10 rovers, 20 cycles, 10 seeds, drifting reliability. Kept is the mean fraction of candidate edges retained. Trust metrics compare learned trust to hidden ground-truth reliability.

pruner	success	solve (ms)	kept	Spearman	AUC
full graph	36.8%	0	100%	0.228	0.608
threshold	38.2%	0	84%	0.218	0.592
top- K	38.2%	0	50%	0.205	0.624
QUBO (ours)	33.8%	1216	55%	0.260	0.662

7 Experimental Evaluation

All experiments run on the open harness in the repository (`battle_test.py`, `simulation.py`, `scale_runner.py`; 486 checks passing) on an AMD Ryzen 7 5800H (16 threads, 8 GB RAM, Python 3.14, NumPy with the reference CBLAS). The simulator is a synthetic dynamic graph world inspired by rover networking; it is not a wireless channel model, and we report it as such.

7.1 Pruning baselines and the honest pruner table

Table 1 compares the QUBO pruner against three baselines over identical scenarios (10 rovers, 20 cycles, 10 seeds, drifting reliability). Two results deserve emphasis. First, mission success now depends on the pruned topology by construction (delivery over the kept graph), so the table measures a real causal effect. Second, the QUBO pruner *loses* on raw success: -4.3 points against the threshold pruner, paired bootstrap 95% CI $[-7.0, -1.5]$, permutation $p = 0.028$. The QUBO’s edge-local utility optimum removes redundancy that bottleneck-path delivery needs; keeping 84 to 100% of edges wins on delivery at this scale. What the QUBO buys is not success but guarantees: degree caps, a hard budget, and exact connectivity of the mission set, which the baselines cannot promise. Its trust estimates are also the best of the table (Spearman 0.260, AUC 0.662). Whether the guarantee is worth 4 success points is a deployment decision, and we report it as one.

7.2 Assignment ablations with paired statistics

To isolate what the GNN contributes, we compare four assignment policies on identical scenarios: random among eligible rovers, trust-only (highest trust, the bandit-style baseline), a no-graph logistic readout on raw node features, and the full trust-aware GNN (Table 2). The GNN beats trust-only by 8.1 points with a paired bootstrap CI excluding zero and a significant permutation test; it beats random by 13.8 points. The margin of the full GNN over the no-graph readout is $+1.7$ points but not significant ($p = 0.74$): at ten-rover density, most of the value is in the trust

Table 2: Assignment ablation, 10 rovers, 20 cycles, 8 seeds. Differences are paired per seed; CI is a 95% paired bootstrap interval and p a sign-flip permutation test.

policy	success
random	26.0%
trust-only	31.7%
no-graph readout	38.1%
trust-aware GNN (ours)	39.8%
GNN – trust-only	+8.1pp, CI [+4.2, +11.7], $p = 0.016$
GNN – no-graph	+1.7pp, CI [−2.1, +6.9], $p = 0.736$

Table 3: Cloud amortization, 10 rovers, 20 cycles, 6 seeds. Solve time is amortized over cycles; the stale decision is intersected with the live graph.

solve interval k	success	solves/cycle	amortized solve (ms)
1	30.3%	1.00	1279
2	29.4%	0.50	618
4	26.9%	0.25	273
8	31.9%	0.15	151

model and the path-aware gating, and message passing adds a small, not yet significant, extra. We consider this the honest answer to “is the GNN necessary”: the graph helps, modestly, and the claim at this scale is limited to that.

7.3 Staleness and cloud amortization

A 1.2s solve cannot run synchronously on a fog device in a fast-changing network, so the architecture amortizes: solve every k cycles (cloud), reuse the stale decision (intersected with the live graph) between solves. Table 3 shows that reusing a two-cycle-old decision costs under one success point while halving solve load; at $k = 4$ and above the measurement is dominated by seed variance, which we say rather than drawing a curve through noise. The guarantee bookkeeping matters here: the simulator counts how often each relaxation level is used (full constraints, no budget, budget only, unconstrained, stale reuse), so the coverage of the connectivity guarantee is measurable rather than asserted.

7.4 Trust quality, training, and faults

Learned trust tracks hidden reliability at Spearman 0.21 to 0.26 and classifies above-median rovers at AUC 0.59 to 0.66 during operation; the QUBO-pruned arm achieves the best values. Training the readout on the unbiased exploration subset (78 samples) does not improve held-out success over fixed weights (36.7% vs 32.8%, difference -3.9 points, CI $[-11.7, +11.7]$, $p = 0.49$): with a selection-bias-free but small dataset, the trained readout is not better, and we report the null result rather than tune until it looks good. Under the scripted fault scenario (most reliable rover degraded plus a packet-loss spike), success returns to its pre-fault baseline within 3 to 4 cycles after the fault expires, with edge lifecycle tracking (active, suspect, probation, removed) keeping pruned links observable.

7.5 Sensitivity

The projected annealer’s exact-solve rate is 87% on a fixed instance set and stays at 87% under every perturbation we tested: trust weight scaled $1.5\times$, uniform utility weights, penalties divided

Table 4: Scaling on range-limited graphs. Connectivity is enforced only for $n \leq 20$ (marked †); we do not present one curve across the transition because the problem changes there. E is soft energy (lower is better). MILP values are exact optima from CBC.

n	$ E $	vars	solve (s)	E_{proj}	E_{greedy}	E_{MILP}
$10^\dagger$	30	427	12.6	-6.39	—	-6.39
$20^\dagger$	63	1076	41.4	-15.05	-11.76	-15.36
30	95	194	15.4	-21.12	-16.62	—
50	169	328	94.5	-36.71	-24.30	—

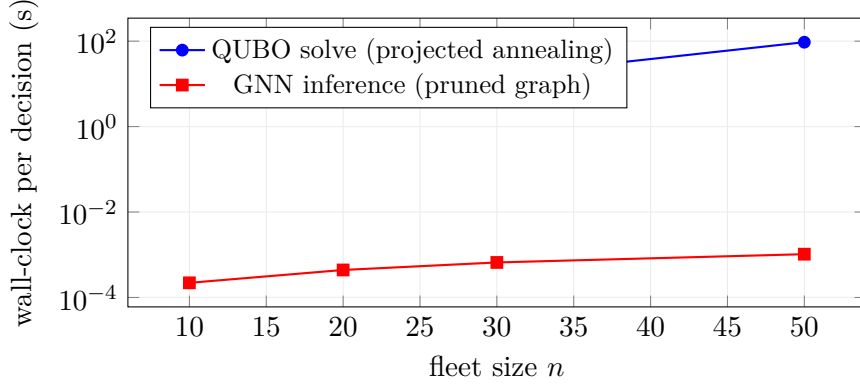


Figure 5: Per-decision wall-clock. The QUBO solve dominates by five orders of magnitude, which is exactly why solving belongs in the cloud tier.

by 5 or multiplied by 5, sparsity penalty raised to 0.3, and stability penalty raised to 0.6. The solver is not fragile to the encoding’s free parameters, which matters because those parameters are currently chosen by hand.

8 Discussion and Limitations

Three findings cut against the naive version of our own pitch, and we consider them the most useful part of the study. First, penalty-encoded constrained QUBOs are easy to write and hard to sample: three annealing variants solve barely half of verifiable instances, so papers reporting such encodings without a solver-level audit risk reporting formulation errors as model behavior. Second, on classical hardware the whole QUBO detour is strictly dominated by direct MILP, which is exact and 100 times faster; the encoding’s justification is hardware targeting, not classical performance. Third, at small fleet sizes the pruner costs far more than the inference it protects and does not beat a threshold pruner on raw success; the architecture is justified by guarantees and by scale, not by small-scale latency or accuracy.

Limitations. Statistical power is limited (8 to 10 seeds per arm); the message-passing margin is not significant; the simulator is synthetic, with no wireless channel model, no hardware-in-the-loop, and no quantum hardware evaluation (the Q matrix is exported but untested on a QPU); the dense Q matrix bounds the practical fleet size at tens of rovers; the scaling study enforces connectivity only up to 20 rovers and does not pretend otherwise; the trained-readout experiment is a null result on limited data; and the utility weights and penalties are hand-chosen, with sensitivity analysis but no principled tuning procedure.

9 Conclusion

We built and audited a pipeline that couples constrained QUBO graph pruning with trust-aware GNN inference for dynamic rover networks. The encoding is derived cleanly and validated exhaustively on finite instances. Once the solver respects feasibility, the formulation is solved exactly on every instance small enough to enumerate, and to within 10^{-3} of exact MILP optima on larger ones, while the direct MILP remains the better classical solver. End-to-end, trust-aware assignment significantly improves mission success over random and trust-only baselines in a simulator where success depends causally on the pruned topology. Equally important, the audit produced three negative results with real content: annealing samplers struggle on penalty-encoded constraints, MILP dominates the QUBO on classical hardware, and the pruner trades a few success points for hard guarantees at small scale. The codebase, harness, and all reported numbers are open and reproducible.

References

- [1] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *ICLR*, 2017.
- [2] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” *NeurIPS*, 2017.
- [3] F. Glover, G. Kochenberger, and Y. Du, “A tutorial on formulating and using QUBO models,” *arXiv:1811.11538*, 2019.
- [4] A. Lucas, “Ising formulations of many NP problems,” *Frontiers in Physics*, 2:5, 2014.
- [5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, 220(4598), 1983.
- [6] D-Wave Systems, “Ocean SDK and dwave-neal,” software documentation, <https://docs.ocean.dwavesys.com/>.
- [7] J. Forrest et al., “COIN-OR branch and cut (CBC),” <https://github.com/coin-or/Cbc>.
- [8] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*, Prentice Hall, 1993.
- [9] G. A. Korsah, A. Stentz, and M. B. Dias, “A comprehensive taxonomy for multi-robot task allocation,” *IJRR*, 32(12), 2013.
- [10] P. Auer, N. Cesa-Bianchi, and P. Fischer, “Finite-time analysis of the multiarmed bandit problem,” *Machine Learning*, 47, 2002.
- [11] D. A. Spielman and S.-H. Teng, “Spectral sparsification of graphs,” *SIAM J. Comput.*, 40(4), 2011.
- [12] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog computing and its role in the internet of things,” *MCC Workshop*, 2012.
- [13] J. Sabater and C. Sierra, “Review on computational trust and reputation models,” *Artificial Intelligence Review*, 24(1), 2005.